AAE-E2001 - Computational Fluid Dynamics D

Assignment 4: Backward-Facing Step

Mohammad-Reza Alibakhshian

March 2026

Theory Exercise

Consider the one–dimensional convection–diffusion equation

$$\frac{\partial c}{\partial t} + u\frac{\partial c}{\partial x} = \Gamma\frac{\partial^2 c}{\partial x^2} \tag{1}$$

The objective is to construct discrete operators D and L such that the time advancement of the solution can be written in the form

$$c^{n+1} = c^n + \Delta t\left(-uDc^n + \Gamma Lc^n\right) \tag{2}$$

Spatial discretization

Assume a uniform grid with spacing Δx and cell values c_i located at the grid points x_i. Using a second–order central difference approximation, the first derivative can be written as

$$\left(\frac{\partial c}{\partial x}\right)_i \approx \frac{c_{i+1} - c_{i-1}}{2\Delta x} \tag{3}$$

for interior points $i = 2, \ldots, N-1$. Similarly, the second derivative is approximated using the central difference formula

$$\left(\frac{\partial^2 c}{\partial x^2}\right)_i \approx \frac{c_{i+1} - 2c_i + c_{i-1}}{\Delta x^2}. \tag{4}$$

Boundary conditions

The scalar field satisfies Dirichlet boundary conditions

$$c(0) = T_1, \qquad c(L) = T_2 \tag{5}$$

These boundary values can be incorporated by extending the vector of unknowns with the known boundary values,

$$\mathbf{c} = \begin{bmatrix} T_1 & c_1 & c_2 & \ldots & c_N & T_2 \end{bmatrix}^T \tag{6}$$

For the first and last interior nodes the derivatives are therefore approximated as

$$\left(\frac{\partial c}{\partial x}\right)_1 = \frac{c_2 - T_1}{2\Delta x}, \qquad \left(\frac{\partial c}{\partial x}\right)_N = \frac{T_2 - c_{N-1}}{2\Delta x} \tag{7}$$

Likewise, the Laplacian at the boundary-adjacent nodes becomes

$$\left(\frac{\partial^2 c}{\partial x^2}\right)_1 = \frac{c_2 - 2c_1 + T_1}{\Delta x^2}, \qquad \left(\frac{\partial^2 c}{\partial x^2}\right)_N = \frac{T_2 - 2c_N + c_{N-1}}{\Delta x^2}. \tag{8}$$

Discrete operators

With these approximations, the convection operator D can be written in matrix form as

$$D = \frac{1}{2\Delta x} \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ -1 & 0 & 1 & \dots & 0 \\ 0 & -1 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & 1 \\ 0 & 0 & 0 & -1 & 0 \end{bmatrix} \tag{9}$$

Similarly, the diffusion operator L becomes a tridiagonal matrix

$$L = \frac{1}{\Delta x^2} \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ 1 & -2 & 1 & \dots & 0 \\ 0 & 1 & -2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & 1 \\ 0 & 0 & 0 & 1 & -2 \end{bmatrix} \tag{10}$$

Final update equation

Substituting the discrete operators into the time advancement formula gives the explicit update

$$c^{n+1} = c^n + \Delta t \left(-uDc^n + \Gamma Lc^n\right) \tag{11}$$

which advances the scalar field in time while satisfying the prescribed boundary conditions.

1 Simulation exercise

Case description

The problem considered in this exercise is the laminar flow of air over a backward-facing step. The inlet velocity was fixed to $U = 0.561$ m/s, and the kinematic viscosity of air was taken as $\nu = 1.51 \times 10^{-5}$ m^2/s. Using the inlet channel height $h = 5.2$ mm, the Reynolds number was calculated as

$$Re = \frac{2Uh}{\nu} = 386.38.$$

Since the flow is in the laminar regime, the inlet channel length was estimated from the laminar entrance-length relation $L_e = 0.06\, Re_h\, h$ which gives $L_e = 60.28$ mm. Three cases were prepared in order to study the influence of the outlet location on the solution. The distance between the step and the outlet was chosen as 40 mm, 80 mm, and 110 mm.

Boundary conditions

The left boundary of the domain was defined as the inlet and the right boundary as the outlet. The upper wall, the lower wall, and the step surfaces were treated as no-slip walls.

For velocity, the boundary conditions were chosen as follows:

- inlet: `fixedValue`,
- outlet: `zeroGradient`,
- walls: no-slip condition.

For pressure, zero-gradient boundary conditions were used on all boundaries except at the outlet, where a fixed value condition was applied.

Mesh generation

The computational mesh was generated using three hexahedral blocks. The first block corresponds to the inlet channel upstream of the step, while the second and third blocks describe the upper and lower parts of the expanded channel downstream of the step.

To improve the near-wall resolution, two-sided `simpleGrading` was used in such a way that the cells become finer close to the walls and coarser toward the interior. In addition, `edgeGrading` was used for the two right-side blocks. This was done to keep the cells refined near the walls without unnecessarily refining the middle region, and also to avoid a discontinuous change in cell size between the left and right blocks.

For the case with 40 mm distance between the step and the outlet, the mesh parameters were

$$NX1 = 60, \qquad NX2 = 40, \qquad NY2 = 50$$

The grading parameters were

$$Gx1 = 0.1, \qquad Gx2 = 10$$

and

$$Gy1 = ((0.5\ \ 0.5\ \ Gx2)\ (0.5\ \ 0.5\ \ Gx1))$$

The corresponding block definitions were

```
blocks
(
    hex (1 6 9 2  0 7 8 3)          ($NX1 $NY2 1)
        simpleGrading ($Gx1 $Gy1 1)
    hex (6 12 15 9  7 13 14 8)      ($NX2 $NY2 1)
        edgeGrading ($Gx2 $Gx2 $Gx2 $Gx2 $Gy1 $Gx1 $Gx1 $Gy1 1 1 1 1)
    hex (5 11 12 6  4 10 13 7)      ($NX2 $NY2 1)
        edgeGrading ($Gx2 $Gx2 $Gx2 $Gx2 $Gy1 $Gx2 $Gx2 $Gy1 1 1 1 1)
);
```

For the other cases the same procedure have been used. To sum up, the total number of cells used for the 40 mm step distance is 7000, for 80 mm distance is 20000, and for 100 mm is 25000 cells.

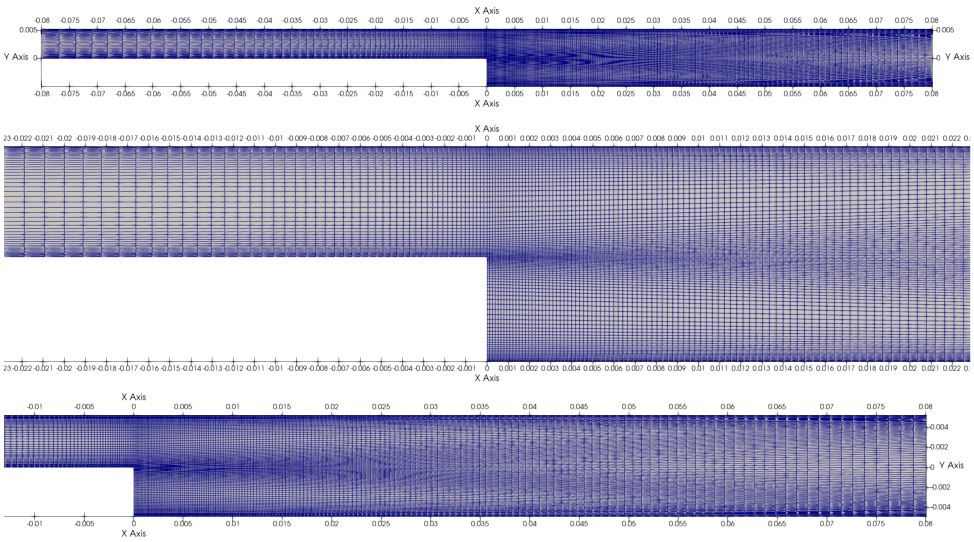


Figure 1: Computational mesh used for the backward-facing step case.

Solver settings

A steady-state solver was used for all simulations. In the discretization settings, the time derivative term in `ddtSchemes` was set to `steadyState`. The gradient terms were discretized using `Gauss linear`, while the convection terms in `divSchemes` were discretized using `bounded Gauss linearUpwind`.

The SIMPLE algorithm was used for pressure-velocity coupling. The relaxation factors were chosen as$\alpha_p = 0.3, \qquad \alpha_U = 0.7$.

To monitor convergence, the `#includeFunc residuals` function was used. The pressure tolerance was set to 10^{-6}, and the velocity tolerance was set to 10^{-8}.

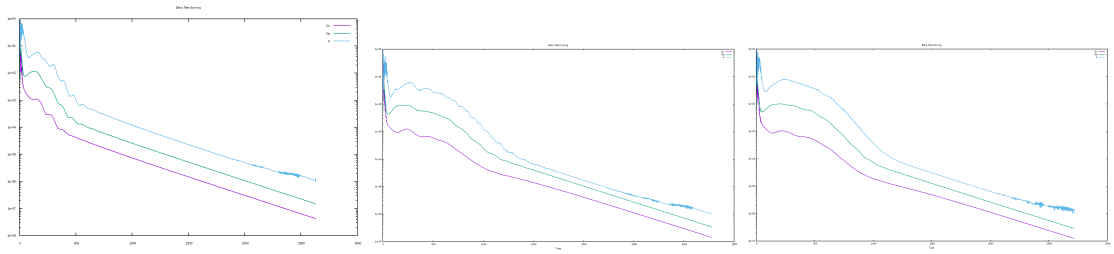


Figure 2: Residual convergence history showing the decay of pressure and velocity residuals during the SIMPLE iterations.

Post-processing

The results were post-processed by extracting velocity profiles of the streamwise velocity component at several downstream locations. These profiles were compared with the reference data of Armaly et al. [1]. In addition, streamline plots and velocity-field contours were generated for all three computational domains. The velocity profiles were evaluated at $x = 0,\ 15.3,\ 24,\ 30.6,\ 35.7,\ 38.8$ mm, which are the locations requested in the assignment.

Results

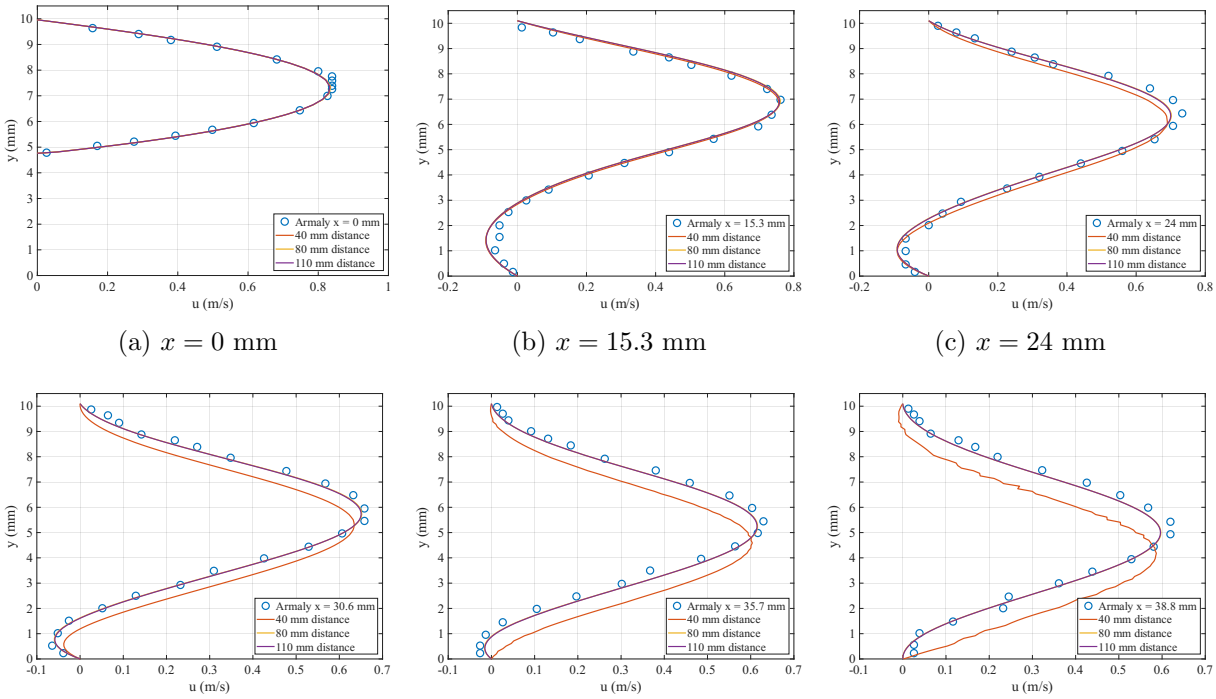
Figures 3a–3f show the velocity profiles at the selected downstream locations. Figures 4 and 5 show the streamlines and velocity fields for the three outlet distances.

Answers to the questions

1. Which solver did you use? Is it a transient or steady-state solver? A steady-state solver was used in this work. The time discretization was defined as `steadyState`, and the pressure–velocity coupling was handled using the SIMPLE algorithm. Therefore, the simulations were carried out as steady calculations rather than transient ones.

2. How does the distance between the outlet and the step affect the results? The outlet distance has a visible effect on the downstream flow development. Close to the step, the velocity profiles from the three cases are relatively similar, but further downstream the case with the shortest outlet distance starts to deviate more from the other two cases and from the reference data. This indicates that when the outlet is placed too close to the step, the flow does not have enough space to develop naturally before reaching the outlet boundary.

For the longer domains, namely 80 mm and 110 mm, the results are closer to each other and also show better agreement with the reference profiles. This suggests that increasing the distance between the step and the outlet reduces the influence of the outlet boundary condition on the recirculation region and on the recovery of the flow.



(a) $x = 0$ mm (b) $x = 15.3$ mm (c) $x = 24$ mm

(d) $x = 30.6$ mm (e) $x = 35.7$ mm (f) $x = 38.8$ mm

Figure 3: Velocity profiles at different streamwise locations.

3. How does zero-gradient boundary condition affect the flow near the boundary? A zero-gradient condition at the outlet means that no streamwise variation is imposed there for the transported quantity. If the outlet is located sufficiently far from the region of interest, this condition works reasonably well because the flow has already developed before reaching the boundary. However, if the outlet is too close to the step, the zero-gradient outlet condition may affect the solution in a non-physical way. In such a case it can alter the development of the recirculation zone and influence the velocity distribution near the outlet.

4. How would the insufficient length of the inlet channel affect flow? If the inlet channel is too short, the flow does not have enough distance to become fully developed before it reaches the step. As a result, the velocity profile at the step is still developing and is different from the expected fully developed laminar channel profile. This would then affect the separation and reattachment behavior downstream of the step, and the numerical results would no longer

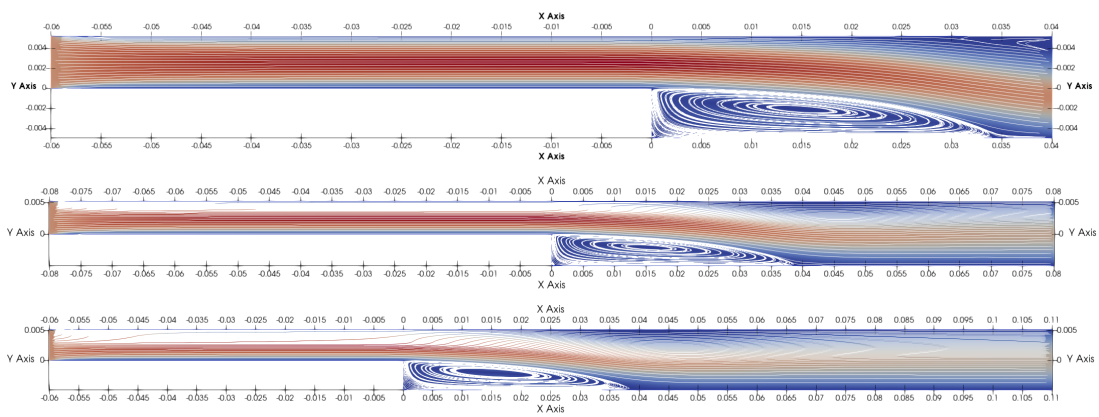


Figure 4: Streamlines for the three outlet distances: 40 mm, 80 mm, and 110 mm.

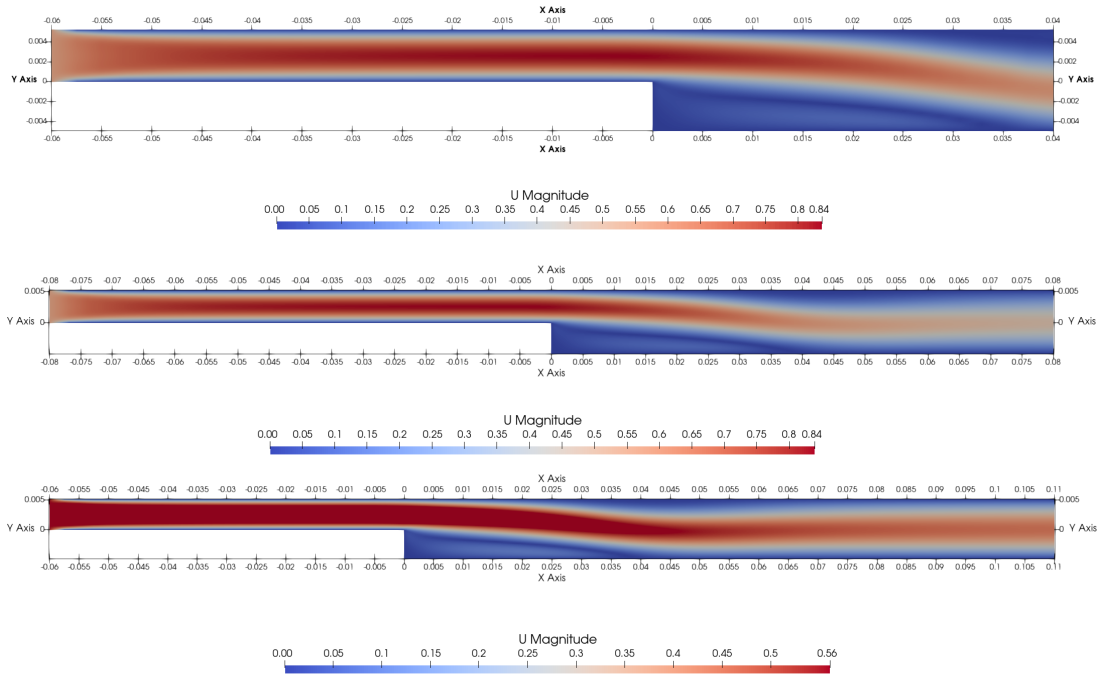


Figure 5: Velocity magnitude contours for the three outlet distances: 40 mm, 80 mm, and 110 mm.

represent the intended backward-facing step case as accurately.

References

[1] B. F. Armaly, F. Durst, J. C. F. Pereira, B. Schönung, Experimental and theoretical investigation of backward-facing step flow, Journal of Fluid Mechanics 127 (1983) 473–496. doi:10.1017/S0022112083002839.

2 Appendix 1: Matlab Code for Plotting

Listing 1: MATLAB code for comparing results

```
clc; clear;

%% The default setup
c = [
    0         0.4470    0.7410;  % Blue
    0.8500    0.3250    0.0980;  % Orange
    0.9290    0.6940    0.1250;  % Yellow
    0.4940    0.1840    0.5560;  % Purple
    0.4660    0.6740    0.1880;  % Green
    0.3010    0.7450    0.9330;  % Light Blue
    0.6350    0.0780    0.1840;  % Red
    0.75      0.75      0;       % Olive
    0.75      0         0.75;    % Magenta
    0.25      0.25      0.25;    % Dark Gray
    1         0.6       0.6;     % Light Pink
    0         0.5       0;       % Dark Green
    0.5       0.5       0.5;     % Gray
    1         0.41      0.16;    % Dark Orange
```

```
    0         0         0        % Black
    ];
set(groot, ...
    'DefaultAxesFontName',       'Times New Roman', ...
    'DefaultTextFontName',       'Times New Roman', ...
    'DefaultLegendFontName',     'Times New Roman', ...
    'DefaultAxesColorOrder',      c,              ...
    'DefaultAxesXGrid',          'on',            ...
    'DefaultAxesYGrid',          'on',            ...
    'DefaultAxesZGrid',          'on',            ...
    'DefaultAxesBox',            'on',            ...
    'DefaultLegendFontSize',      22,             ...
    'DefaultAxesFontSize',        22,             ...
    'DefaultLineLineWidth',       2, ...
    'DefaultAxesGridAlpha',0.1, ...
    'DefaultFigurePosition',[100 100 1000 730]);

mrkrs = {'o','s','^','d','v','>','<','p','h','x','+'};
lineStyles = {'-','--','-.',':','-'};

%% Input varialbes
U = 0.561;                  % m/s
h = 5.2;                    % mm
h = h * 1E-3;               % to meters
H = 10.1;                   % mm
H = H * 1E-3;               % to meters
nu = 1.51E-5;               % m2/s kinetamtic viscosity of air at 20C
fprintf('Kinematic viscosity is: %.2e\n',nu);
Re = 2 * U * h / nu;        % Reynolds Number -- found out that this is LAMINAR (Re<2300)
fprintf('The flow Reynolds Number is: %.2f\n',Re);
Re_h = U * h / nu;
L_e =  0.06 * Re_h * h;   % for laminar flows Le/d = 0.06 * Re_d
fprintf('The entrance length for laminar flow is L_e = 0.06 * Re_h * h, and here is: %.2f mm\n ',L_e*1E3);

%% Read Table at 0 and the corresponding calculations
armaly0 = readtable("Armaly0mm.dat");
armaly0.Properties.VariableNames = {'y','u'};

u0_40 = readtable('40m\u0.dat');
u0_40.Properties.VariableNames = {'y','u','v','w'};
u0_40.y = u0_40.y*1e3 + 4.76;

u0_80 = readtable('80m\u0.dat');
u0_80.Properties.VariableNames = {'y','u','v','w'};
u0_80.y = u0_80.y*1e3 + 4.76;

u0_110 = readtable('110m\u0.dat');
u0_110.Properties.VariableNames = {'y','u','v','w'};
u0_110.y = u0_110.y*1e3 + 4.76;

figure(1); clf; hold on;
plot(armaly0.u,armaly0.y,'o','DisplayName','Armaly x = 0 mm','MarkerSize',12);
plot(u0_40.u,u0_40.y,'DisplayName','40 mm distance');
plot(u0_80.u,u0_80.y,'DisplayName','80 mm distance');
plot(u0_110.u,u0_110.y,'DisplayName','110 mm distance');

```

```
legend('Location','best'); hold off;
xlabel('u (m/s)'); ylabel('y (mm)');ylim([0 10.5]);
legend('Location','southeast');
exportgraphics(gcf,'Fig1.jpg');
exportgraphics(gcf,'Fig1.pdf','ContentType','vector');

%% Read Table at other times

armalyFiles = { ...
    'Armaly15.3mm.dat'
    'Armaly24mm.dat'
    'Armaly30.6mm.dat'
    'Armaly35.7mm.dat'
    'Armaly38.8mm.dat'};

uid  = [15 24 30 35 38];
dist = [40 80 110];

for k = 1:length(armalyFiles)

    armaly = readtable(armalyFiles{k});
    armaly.Properties.VariableNames = {'y','u'};

    figure(k+1); clf; hold on

    plot(armaly.u,armaly.y,'o',...
        'DisplayName',sprintf('Armaly x = %s mm',armalyFiles{k}(7:end-6)),...
        'MarkerSize',12)

    for j = 1:length(dist)

        filename = sprintf('%dm\\u%d.dat',dist(j),uid(k));
        T = readtable(filename);

        T.Properties.VariableNames = {'y','u','v','w'};
        T.y = T.y*1e3 + 4.9;

        plot(T.u,T.y,'DisplayName',sprintf('%d mm distance',dist(j)))

    end

    xlabel('u (m/s)')
    ylabel('y (mm)')
    ylim([0 10.5])
    legend('Location','southeast')

    exportgraphics(gcf,sprintf('Fig%d.jpg',k+1))
    exportgraphics(gcf,sprintf('Fig%d.pdf',k+1),'ContentType','vector')

end
```